

Software Architecture and Design Specification

Project: File Compression Tool (Basic ZIP Implementation)

Version: 1.0

Authors: Adarsh Prakash (PES2UG24CS027), Aakash Agarwal (PES2UG24CS009), Abhijna Marathe (PES2UG24CS020), Abhirup Roy (PES2UG24CS025)

Date: 14-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Project Team	Initial SAD document derived from submitted SRS

Approvals

Role	Name	Signature/Date
Course Coordinator	<Name>	<Signature / Date>

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the File Compression Tool, a basic ZIP-compatible archive utility. It translates the functional requirements, non-functional requirements, and security requirements defined in the project's Software Requirements Specification (SRS) into a concrete component structure, technology choices, interaction flows, and interface design that the team will implement.

1.2 Scope

This document covers the architecture and design of a command-line tool that compresses selected files and folders into a ZIP archive, extracts an existing ZIP archive to a chosen destination, lists archive contents without extracting, validates archive structure, verifies entry integrity, and safely rejects unsafe extraction paths. It does not cover GUI design beyond basic CLI usability, cloud storage integration, distributed processing, multi-volume archives, or encryption, which are out of scope for this basic implementation.

1.3 Audience

Developers, QA engineers, the project team, instructors, and assessment evaluators.

1.4 Definitions

SRS, ZIP, CRC, CLI, API, I/O, SHA-256, NFR, FR, RTM, path traversal.

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables for the project: a component diagram, component descriptions, the chosen architecture pattern and rationale, technology stack, risk analysis, requirements traceability, a threat model, sequence diagrams for the two primary flows, module-level interface (API) design, error handling and logging strategy, UX notes, and open issues.

2.2 Related Documents

Software Requirements Specification (SRS) — File Compression Tool (Basic ZIP Implementation), v1.0, submitted 05-09-2026.

3. Architecture

3.1 Goals & Constraints

Goals:

- Correctness — a compress-extract round trip must preserve file size and SHA-256 content hash for supported files (FCT-NF-004).
- Simplicity — the design should stay as small and easy to reason about as the SRS scope allows, avoiding unnecessary components or layers.
- Safety — the tool must never write outside a user-selected destination directory (FCT-SR-001, FCT-F-008).
- Portability — the tool must behave correctly across Windows, Linux, and macOS path conventions (FCT-NF-005).

Constraints:

- Implemented in Python 3, using only the standard library (zipfile, hashlib, argparse, pathlib, logging) — no external dependencies, keeping setup and grading simple.
- Command-line interface only; no GUI in this iteration.
- Must not assume all input files fit in memory at once — files are streamed rather than fully buffered where the standard library allows.
- Performance target: 95th-percentile elapsed time ≤ 2 seconds per 100 MB (FCT-NF-001); peak memory ≤ 512 MB for a 1 GB workload (FCT-NF-002).

3.2 Stakeholders & Concerns

Stakeholder	Concerns
User	Simple, predictable commands; clear progress and error messages
Project Team (Developers)	Modular, testable code; easy to demo and explain in a viva
Assessment Evaluator	Clear traceability to SRS requirements; working, verifiable demo
Security (implicit stakeholder)	No path traversal, no silent data loss, no sensitive data in logs

3.3 Component (UML) Diagram

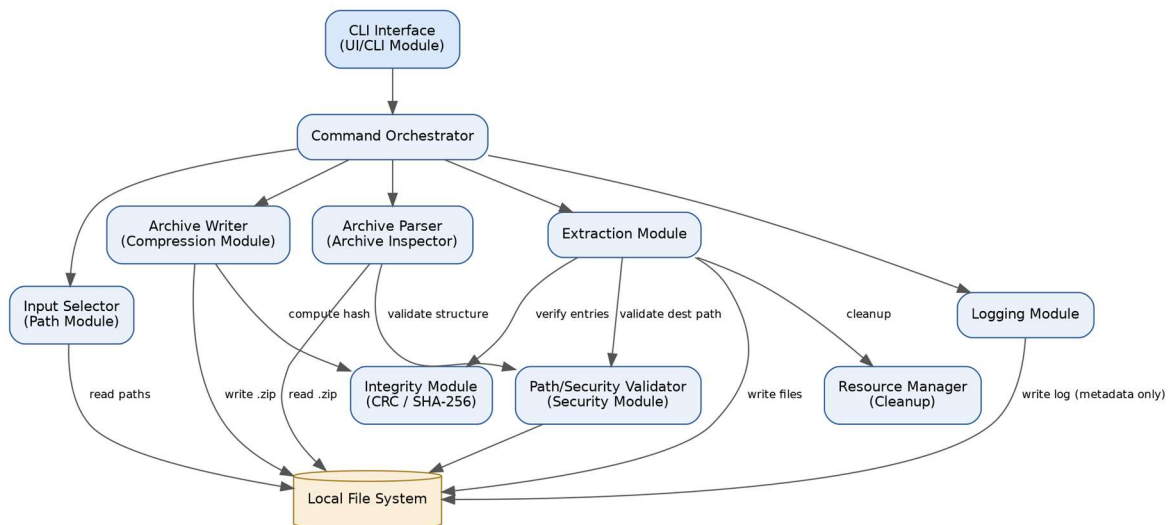


Figure 1: Component diagram of the File Compression Tool

3.4 Component Descriptions

Component	Responsibility	SRS Module
CLI Interface	Parses user commands (compress, extract, list) and arguments; displays progress, status, and errors	UI/CLI Module
Command Orchestrator	Coordinates the call sequence between modules for a given command	—
Input Selector	Validates and normalizes selected file/folder paths; computes relative paths for directory structure preservation	Path Module
Archive Writer	Creates the ZIP archive and writes entries using Python's zipfile module	Compression Module / Archive Writer
Archive Parser	Reads and validates ZIP structure; lists entries without extracting	Archive Parser / Archive Inspector

Component	Responsibility	SRS Module
Extraction Module	Extracts entries to the destination, applying the overwrite/skip/rename policy	Extraction Module
Path/Security Validator	Resolves every entry path and rejects any that escape the destination directory; validates archive structure before processing	Security Module
Integrity Module	Computes and verifies CRC (via zipfile) and SHA-256 hashes for round-trip integrity checks	Integrity Module
Resource Manager	Cleans up temporary files after success or failure	Resource Manager
Logging Module	Records operation metadata (operation, path, result) — never file contents	Logging Module

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is used, with four layers: (1) Presentation — the CLI Interface; (2) Application/Orchestration — the Command Orchestrator, which sequences calls for each command; (3) Domain/Core Logic — Archive Writer, Extraction Module, Archive Parser, Integrity Module, and Path/Security Validator; and (4) Data Access — the local file system, accessed through Python's standard library.

This pattern was chosen because the tool is a single-process, local, desktop CLI utility with no networked services or concurrent users, so a microservices or event-driven pattern would add complexity without benefit. A layered architecture keeps responsibilities cleanly separated (path handling, compression, security, integrity), which directly supports the SRS's modular traceability (each requirement maps to one module) while staying simple enough for a four-person team to build, test, and explain.

3.6 Technology Stack & Data Stores

- Language: Python 3.x
- Compression/archive handling: zipfile (standard library) — DEFLATE-compatible ZIP read/write
- Integrity: hashlib (SHA-256); zipfile's built-in CRC-32 per entry
- CLI parsing: argparse
- Path handling: pathlib / os (for cross-OS normalization, FCT-NF-005)
- Logging: logging (standard library), writing operational metadata only (FCT-SR-004)
- Data store: no database — the ZIP archive itself and the local file system are the persistent data stores; an optional local log file holds diagnostic metadata

3.7 Risks & Mitigations

Risk	Mitigation
Large files exceeding available memory	Stream file reads/writes via zipfile's chunked I/O rather than loading entire files into memory
Path traversal via malicious archive entries (“zip-slip”)	Resolve every extraction path and verify it remains within the destination directory before writing (FCT-SR-001)
Corrupted or malformed archives crashing the tool	Validate ZIP structure up front; catch and report exceptions with clear, actionable messages (FCT-F-009, FCT-NF-003)
Cross-platform path separator issues	Use pathlib for all path construction and normalization (FCT-NF-005)
Silent data loss on duplicate output filenames	Require an explicit overwrite/skip/rename policy before proceeding; default to non-destructive behavior (FCT-SR-005)

3.8 Traceability to Requirements

The table below maps key SRS requirements to the architectural component responsible for satisfying them (consistent with the Requirements Traceability Matrix in the SRS).

Req ID	Requirement (short)	Component
FCT-F-001	Select compression inputs	Input Selector
FCT-F-002	Create ZIP archive	Archive Writer
FCT-F-003	Preserve directory structure	Input Selector / Archive Writer
FCT-F-004	Compress file data	Archive Writer
FCT-F-005	Report compression status	CLI Interface
FCT-F-006	Select extraction destination	CLI Interface
FCT-F-007	Extract archive entries	Extraction Module
FCT-F-008	Block path traversal	Path/Security Validator
FCT-F-009	Reject malformed archive	Archive Parser
FCT-F-010	List archive contents	Archive Parser
FCT-F-011	Apply duplicate-file policy	Extraction Module
FCT-F-012	Verify entry integrity	Integrity Module
FCT-F-013	Preserve binary contents	Archive Writer / Extraction Module
FCT-F-014	Clean temporary resources	Resource Manager
FCT-F-015	Operation completion summary	CLI Interface
FCT-SR-001	Path traversal protection	Path/Security Validator
FCT-SR-002	ZIP structure validation	Archive Parser
FCT-SR-003	Entry integrity verification	Integrity Module
FCT-SR-004	Sensitive-data-safe logging	Logging Module
FCT-SR-005	Non-destructive overwrite policy	Extraction Module

3.9 Security Architecture

Threat modeling (STRIDE), adapted for a local, single-user, offline CLI tool:

- Spoofing — not applicable; the tool has no authentication or network identity to spoof.
- Tampering — mitigated by entry-level CRC-32 (built into ZIP) and SHA-256 round-trip verification via the Integrity Module.
- Repudiation — the Logging Module records each operation (type, path, result) so actions can be reviewed after the fact.
- Information Disclosure — all processing is local (no network transmission of file contents); logs contain operational metadata only, never file contents (FCT-SR-004).
- Denial of Service — memory-bounded, streamed processing avoids resource exhaustion from very large archives (FCT-NF-002).
- Elevation of Privilege — the tool runs with the invoking user's own file-system permissions only; extraction is confined to the selected destination directory via path validation (FCT-SR-001).

4. Design

4.1 Design Overview

The tool follows its layered architecture end-to-end: the CLI Interface parses a command and delegates to the Command Orchestrator, which calls the relevant core module(s) (Archive Writer for compression, Archive Parser and Extraction Module for extraction/listing), which in turn use the Path/Security Validator and Integrity Module as needed, and read/write through the local file system layer. Each module has a single, narrow responsibility so it can be implemented and tested independently by different team members.

4.2 UML Sequence Diagrams

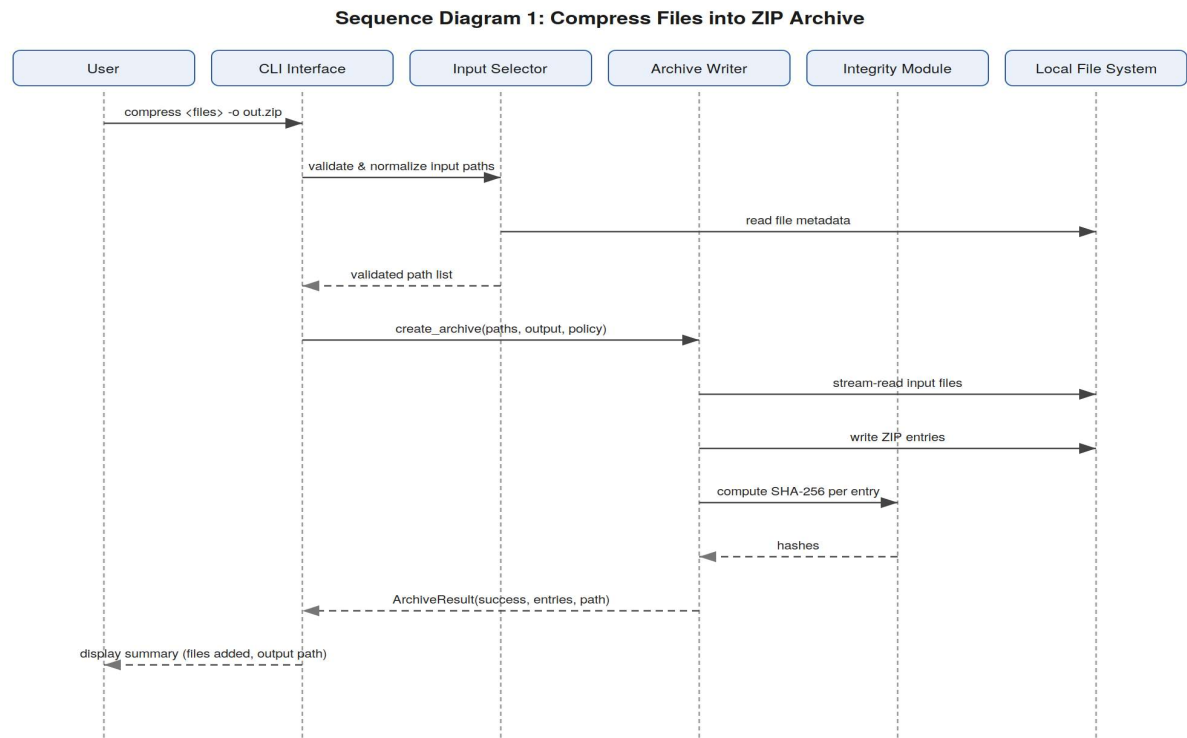


Figure 2: Sequence diagram — compressing files into a ZIP archive

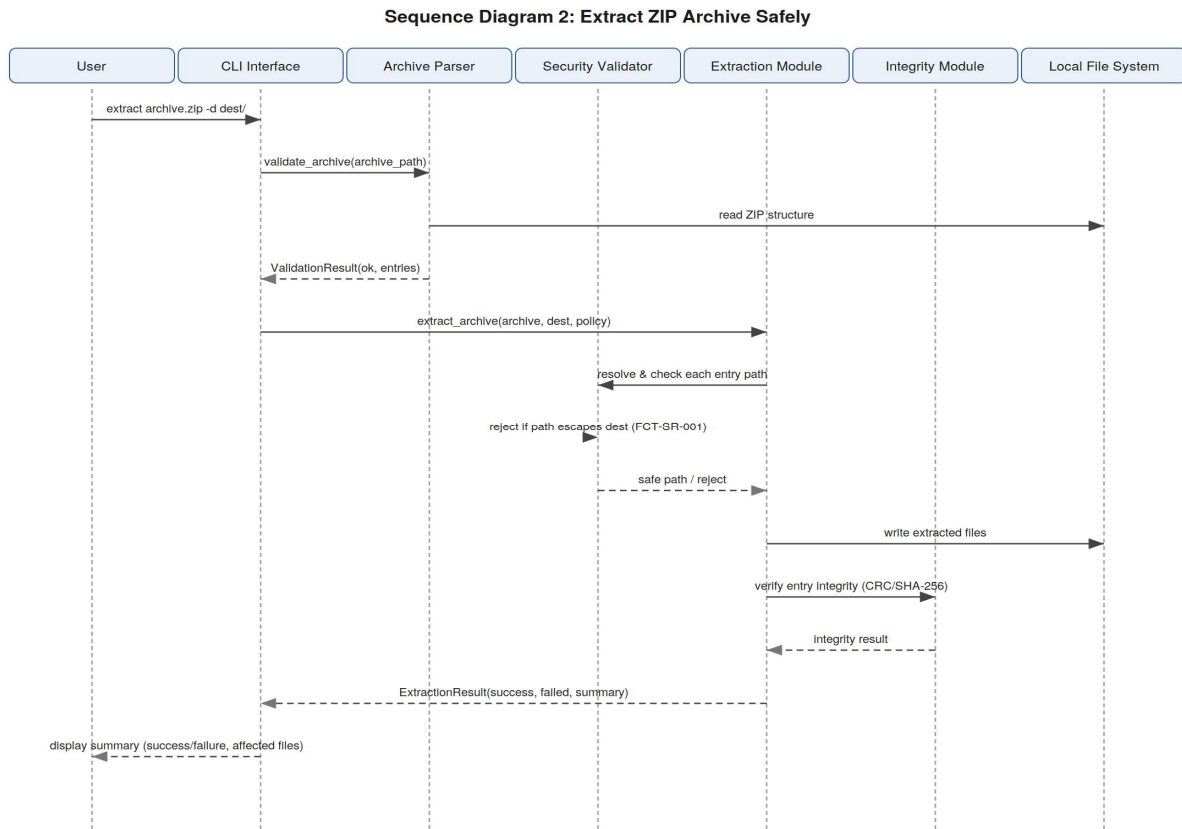


Figure 3: Sequence diagram — extracting a ZIP archive safely

4.3 API Design

As a local CLI tool, the “API” consists of module-level function interfaces rather than network endpoints. Interfaces for the two core components are defined below.

Component: Archive Writer

- Function: `create_archive(input_paths: List[Path], output_path: Path, overwrite_policy: str) → ArchiveResult`
- Request: list of input file/folder paths, target archive path, overwrite/skip/rename policy
- Response: `ArchiveResult { success: bool, entries_written: int, output_path: Path, errors: List[str] }`
- Errors: `FileNotFoundError` (missing input), `PermissionError` (no write access, FCT-F-002 dependency), `ValueError` (empty input list)

Component: Extraction Module

- Function: `extract_archive(archive_path: Path, destination: Path, overwrite_policy: str) → ExtractionResult`
- Request: archive path, destination directory, overwrite/skip/rename policy
- Response: `ExtractionResult { success: bool, extracted: List[str], skipped: List[str], failed: List[str] }`
- Errors: `PathTraversalError` (entry resolves outside destination, FCT-SR-001), `BadZipFile` (corrupted/malformed archive, FCT-F-009), `PermissionError`

Component: Archive Parser

- Function: `list_entries(archive_path: Path) → List[EntryInfo]`
- Function: `validate_archive(archive_path: Path) → ValidationResult { ok: bool, reason: str }`

4.4 Error Handling, Logging & Monitoring

Error messages identify both the failed operation and the affected file or path (FCT-NF-003), e.g. “Extract failed: 'data/report.zip' → 'notes/report.txt' would escape destination directory.” Logs record only operational metadata (operation type, timestamp, path, result) and never file contents or archive payloads (FCT-SR-004). For monitoring, the tool tracks operation duration to validate the performance target (FCT-NF-001) and can report peak memory usage during testing (FCT-NF-002).

4.5 UX Design

The CLI exposes three primary commands — `compress`, `extract`, and `list` — each with a `--help` description. Progress for multi-file or long-running operations is shown as plain-text status (e.g. file counts or percentage), satisfying FCT-F-005 without relying on color alone to convey success or failure, so the tool remains usable in plain terminals.

4.6 Open Issues & Next Steps

- Optional GUI wrapper around the same core modules
- Password-protected / encrypted archive support (explicitly out of scope for this basic implementation)
- Multi-volume archive support
- Parallel compression for improved throughput on multi-file inputs

5. Appendices

5.1 Glossary

ZIP, CRC, SHA-256, CLI, path traversal, NFR, FR, RTM.

5.2 References

- Project SRS — File Compression Tool (Basic ZIP Implementation), v1.0
- IEEE 42010 (Architecture Description)
- OWASP guidance on path traversal / zip-slip
- Python zipfile standard library documentation

5.3 Tools

Graphviz and a custom SVG sequence-diagram script for diagrams; Python 3 standard library for implementation.